

Visualisierung kinematischer Kennwerte

Philipp Mikos

April 2025

Inhaltsverzeichnis

1	Einleitung	2
2	Motivation	3
3	Ziel	3
4	Theoretischer Hintergrund	4
4.1	Darstellung der Position und Orientierung eines Roboters	4
4.2	Euler-Winkel und Roll-Pitch-Yaw-Winkel	5
4.3	Rotationsmatrizen	6
4.4	Quaternionen und Unit-Quaternionen	7
4.5	Differentialantrieb	8
4.6	JupyterLab	9
5	Anforderungen	9
6	Aufbau des Frameworks	11
6.1	System- und Softwareumgebung	11
6.2	Framework-Library (Implementierungsdetails)	16
6.3	Beispiel-Notebooks	17

6.3.1	robot.ipynb	17
6.3.2	rotations.ipynb	19
6.3.3	quader.ipynb	20
7	Entwicklungsprozess	23
8	Evaluation	33
9	Ausblick auf die Bachelorarbeit	35

1 Einleitung

Die Grundlagen der Kinematik stellen einen wesentlichen Bestandteil der Robotik dar und sind sowohl für mobile als auch für stationäre Roboter von zentraler Bedeutung. Während es bei stationären Robotern häufig darum geht, die Position des Endeffektors präzise zu berechnen, konzentriert sich die Kinematik bei mobilen Robotern vor allem darauf, deren Position zuverlässig bestimmen zu können. Ein fundiertes Verständnis der erforderlichen kinematischen Modelle ist unerlässlich, um diese Größen korrekt zu berechnen. [SNS11, S. 57]

Im Rahmen dieser Projektarbeit wird ein Framework entwickelt, das die Visualisierung kinematischer Kennwerte ermöglicht. Das Ziel dieses Frameworks ist es, unterstützend in der Lehre eingesetzt zu werden, um das Verständnis der Kinematik zu fördern und die praktische Anwendung dieser Konzepte zu erleichtern. Ein besonderer Fokus dieser Arbeit liegt auf der Rotation von Körpern im Raum. Dabei wird insbesondere erläutert, was unter Eulerwinkeln sowie Roll-Pitch-Yaw-Winkeln zu verstehen ist und worin die Unterschiede zwischen beiden Darstellungen bestehen. Darüber hinaus werden auch Rotationsmatrizen sowie Quaternionen behandelt, um die verschiedenen Ansätze zur Beschreibung von Rotationen umfassend darzustellen.

2 Motivation

Obwohl kinematische Modelle in vielen Bereichen der Robotik eine zentrale Rolle spielen, fällt es Studierenden häufig schwer, diese abstrakten mathematischen Konzepte vollständig zu durchdringen. Besonders Rotationen im dreidimensionalen Raum – etwa durch Eulerwinkel, Roll-Pitch-Yaw-Darstellungen, Rotationsmatrizen oder Quaternionen beschrieben – wirken in der Theorie oft wenig anschaulich und sind ohne visuelle Unterstützung schwer intuitiv erfassbar [Gut+16, S. 94–95].

In der Hochschullehre entsteht dadurch eine Lücke zwischen theoretischem Wissen und praktischem Verständnis. Lehrende stehen vor der Herausforderung, komplexe Zusammenhänge nicht nur formal korrekt, sondern auch nachvollziehbar und greifbar zu vermitteln. Gleichzeitig benötigen Studierende Möglichkeiten, das Gelernte interaktiv zu erproben, um eine tiefere, anwendungsorientierte Intuition für die Kinematik zu entwickeln.

Moderne Lehrmethoden, die auf Visualisierung und Interaktion setzen, können hier entscheidend zur Verbesserung des Lernerfolgs beitragen. Insbesondere browserbasierte und plattformunabhängige Lösungen, wie sie in JupyterLab realisierbar sind, eröffnen neue Perspektiven für eine zeitgemäße und praxisnahe Vermittlung technischer Inhalte [Gut+16, S. 239–240, 407].

3 Ziel

Vor diesem Hintergrund zielt das Projekt darauf ab, ein interaktives Visualisierungs-Framework zu entwickeln, das die Lehre im Bereich der Robotik an der Fachhochschule Dortmund unterstützt und bereichert. Das Framework wird speziell für den Einsatz innerhalb der JupyterLab-Umgebung konzipiert und soll dazu dienen, kinematische Zusammenhänge auf intuitive und anschauliche Weise

darzustellen.

Ein zentrales Anliegen ist es, Lehrpersonen und Aufgabenstellern ein Werkzeug an die Hand zu geben, mit dem sie durch einfache Python-Befehle individualisierbare Aufgabenstellungen und Übungsszenarien gestalten können. Diese sollen es Studierenden ermöglichen, eigenständig mit Parametern zu experimentieren, Bewegungsabläufe zu analysieren und dadurch ein tieferes Verständnis für die Wirkungsweise kinematischer Modelle zu entwickeln.

Besonderes Augenmerk liegt auf der Darstellung verschiedener Rotationsmodelle im Raum, darunter Eulerwinkel, Roll-Pitch-Yaw-Winkel, Rotationsmatrizen und Quaternionen. Durch deren visuelle Umsetzung werden nicht nur Unterschiede und Gemeinsamkeiten der jeweiligen Darstellungsformen sichtbar, sondern auch deren konkrete Bedeutung im räumlichen Kontext erfahrbar gemacht.

Insgesamt verfolgt das Projekt das Ziel, den Lernprozess im Bereich der Kinematik durch moderne, interaktive Lehrmittel zu unterstützen und einen Beitrag zu einer zeitgemäßen, verständnisorientierten Ausbildung in der Robotik zu leisten.

4 Theoretischer Hintergrund

4.1 Darstellung der Position und Orientierung eines Roboters

Ein Roboter wird in der Regel als ein unverformbarer, fester Körper (Rigid Body) betrachtet, der sowohl eine bestimmte Position als auch eine Orientierung im Raum besitzt. Die Position beschreibt den Abstand und die Verschiebung zwischen dem globalen Weltkoordinatensystem und dem körpereigenen Koordinatensystem des Roboters. Die Orientierung hingegen gibt den Unterschied

in der Winkelausrichtung zwischen dem Weltkoordinatensystem und dem lokalen Koordinatensystem des Roboters an. Die Kombination aus Position und Orientierung eines Körpers wird als Pose bezeichnet. Die Transformation von einer Pose in eine andere kann intuitiv als Bewegung interpretiert werden. Diese Transformation setzt sich aus einer Translation (Verschiebung) und einer Rotation (Drehung) zusammen [Cor23, S. 24–27]. Speziell bei radgetriebenen Robotern wird die Pose auf einer Ebene beschrieben. In diesem Fall besteht die Pose aus den X- und Y-Koordinaten im globalen Koordinatensystem sowie dem Winkel zwischen dem globalen und dem körpereigenen Koordinatensystem. Damit lässt sich die Pose als ein Vektor aus diesen drei Elementen darstellen [SNS11, S. 59].

4.2 Euler-Winkel und Roll-Pitch-Yaw-Winkel

Euler’s Rotationstheorem besagt, dass zwei beliebige unabhängige, orthonormale Koordinatensysteme durch eine Abfolge von maximal drei Rotationen um Koordinatenachsen miteinander in Beziehung gesetzt werden können, wobei keine zwei aufeinanderfolgenden Rotationen um die gleiche Achse erfolgen dürfen [Kui99, S. 136]. Das bedeutet, dass jede Rotation zwischen zwei Koordinatensystemen im dreidimensionalen Raum durch eine Sequenz von drei Rotationswinkeln beschrieben werden kann, wobei jeder Winkel einer bestimmten Achse zugeordnet ist. Die Rotationen werden nacheinander ausgeführt: Zunächst erfolgt eine Rotation des Weltkoordinatensystems um eine Achse, wodurch ein neues Koordinatensystem entsteht. Anschließend wird dieses neue System um eine seiner eigenen Achsen rotiert, gefolgt von einer dritten Rotation um eine Achse des wiederum neu entstandenen Systems.

Insgesamt gibt es zwölf eindeutige Rotationsabfolgen. Sechs davon beinhalten Wiederholungen derselben Achse (aber nicht unmittelbar hintereinander),

beispielsweise XYX , XZX , YXY , YZY , ZXZ oder ZYZ . Die anderen sechs Rotationsabfolgen verwenden jeweils alle drei Achsen einmal, beispielsweise XYZ , XZY , YZX , YXZ , ZXY oder ZYX .

Der Begriff Euler-Winkel wird häufig verwendet, um jede Art von dreifacher Rotationsdarstellung zu bezeichnen. Diese Bezeichnung ist jedoch nicht präzise, da – abhängig von der Reihenfolge der Rotationen – verschiedene Darstellungen entstehen. Deshalb muss die genaue Rotationsabfolge angegeben werden, auch wenn innerhalb bestimmter technischer Fachgebiete meist eine stillschweigende Konvention herrscht.

Eine besondere Rolle spielen die sogenannten Roll-Pitch-Yaw-Winkel, bei denen es zwei gebräuchliche Rotationssequenzen gibt: ZYX und XYZ . In der mobilen Robotik wird typischerweise die ZYX -Sequenz verwendet, während bei stationären Roboterarmen oft die XYZ -Sequenz Anwendung findet [Cor23, S. 48–51].

4.3 Rotationsmatrizen

Eine Rotationsmatrix ist eine spezielle Art von Matrix, die ein Koordinatensystem um einen Winkel θ dreht und es dadurch in ein neues Koordinatensystem transformiert. Die Spalten einer Rotationsmatrix können dabei als die transformierten Einheitsvektoren des ursprünglichen Koordinatensystems interpretiert werden.

Rotationsmatrizen sind stets orthonormal, das bedeutet, ihre Spalten (bzw. Zeilen) stehen senkrecht aufeinander und haben die Länge Eins. Daraus folgt die wichtige Eigenschaft:

$$R^{-1} = R^T$$

wobei R^{-1} die Inverse und R^T die Transponierte der Matrix R ist. Ein weiteres charakteristisches Merkmal von Rotationsmatrizen ist, dass ihre Determinante

stets den Wert 1 hat.

Rotationsmatrizen sind außerdem Elemente der sogenannten speziellen orthogonalen Gruppe der Dimension 2, die als $SO(2)$ bezeichnet wird. dies schreiben wir auch als

$$R \in SO(2) \subset \mathbb{R}^{2 \times 2}$$

Die spezielle orthogonale Gruppe ist eine Gruppe bezüglich der Matrixmultiplikation. Das bedeutet, dass das Produkt zweier Rotationsmatrizen wieder eine Rotationsmatrix ist und somit ebenfalls zur Gruppe gehört. Ebenso ist die Inverse jeder Rotationsmatrix ebenfalls Teil der Gruppe [Cor23, S. 33].

4.4 Quaternionen und Unit-Quaternionen

Quaternionen wurden 1843 von Sir William Rowan Hamilton entdeckt. Sie erweitern die komplexen Zahlen und bestehen aus einem Skalar- und einem Vektoranteil:

$$q = s + v_x i + v_y j + v_z k$$

wobei i , j und k besondere Einheiten mit $i^2 = j^2 = k^2 = ijk = -1$ sind. Quaternionen bilden einen Vektorraum: Man kann sie addieren, skalieren, multiplizieren (Hamilton-Produkt) und konjugieren. Die Multiplikation ist nicht kommutativ. Der Betrag eines Quaternions ist:

$$||q|| = \sqrt{s^2 + v_x^2 + v_y^2 + v_z^2}$$

Unit-Quaternionen sind Quaternionen mit Betrag 1. Sie werden häufig verwendet, um Rotationen im Raum darzustellen. Eine Rotation um eine Achse $\hat{v}$ mit Winkel θ wird als

$$q = \cos(\theta/2) + \hat{v} \sin(\theta/2)$$

bezeichnet.

Quaternionen bieten bei der Darstellung von Rotationen mehrere Vorteile. Sie

ermöglichen eine recheneffiziente Kombination von Rotationen und vermeiden dabei Probleme wie den sogenannten Gimbal Lock, der bei der Verwendung von Euler-Winkeln auftreten kann. Zudem sind Quaternionen kompakter als Rotationsmatrizen und benötigen weniger Speicherplatz und Rechenleistung, was sie besonders attraktiv für Anwendungen in Bereichen wie Robotik, Computer Vision, Computergrafik und Navigation macht [Cor23, S. 59–61].

4.5 Differentialantrieb

Ein mobiler Roboter mit Differentialantrieb besitzt zwei angetriebene Räder, jeweils eines auf jeder Seite des Roboters. Größere Roboter können zusätzlich über ein oder mehrere weitere Räder auf jeder Seite verfügen, die entweder passiv sind oder mechanisch mit den aktiven Rädern gekoppelt werden, sodass sie sich einen Motor teilen. Die Räder sind starr am Roboter befestigt.

Die Steuerung des Roboters erfolgt durch die unterschiedliche Antriebsgeschwindigkeit der Räder. Drehen sich beide angetriebenen Räder mit gleicher Geschwindigkeit, bewegt sich der Roboter geradeaus. Rotieren die Räder hingegen mit unterschiedlichen Geschwindigkeiten, führt dies zu einer Drehbewegung des Roboters.

Bei Robotern mit Differentialantrieb wird üblicherweise davon ausgegangen, dass es sich bei den verbauten Rädern um Standardräder handelt. Das bedeutet, dass sich die Räder – und somit der gesamte Roboter – ausschließlich in x-Richtung bewegen können, während Bewegungen oder ein Rutschen in y-Richtung nicht möglich sind. Zudem wird angenommen, dass jedes Rad nur einen einzigen Kontaktpunkt mit der Ebene besitzt [Cor23, S. 140–141] [HLN12, S. 107].

4.6 JupyterLab

JupyterLab ist eine leistungsstarke und erweiterbare Umgebung zur Erstellung und Bearbeitung von sogenannten computational notebooks. Es ist Teil des größeren Project Jupyter, das sich zum Ziel gesetzt hat, Werkzeuge und Standards für das interaktive Arbeiten mit Rechen-Notizbüchern zu entwickeln.

Ein computational notebook ist ein Dokument, das Computer-Code, Freitext-Beschreibungen, Daten, grafische Darstellungen (z.B. Diagramme, 3D-Modelle) sowie interaktive Bedienelemente miteinander kombiniert. In Verbindung mit einem Editor wie JupyterLab ermöglichen diese Notebooks eine schnelle, interaktive Entwicklung, die sich besonders gut für das Prototyping von Code, die Datenexploration, die Visualisierung und das Teilen von Ideen eignet.

JupyterLab ist eng verwandt mit anderen Anwendungen unter dem Dach von Project Jupyter, wie zum Beispiel Jupyter Notebook und Jupyter Desktop. Im Vergleich zu Jupyter Notebook bietet JupyterLab jedoch eine fortschrittlichere, funktionsreichere und individuell anpassbare Benutzererfahrung [Jup24].

5 Anforderungen

Das zu entwickelnde Framework soll eine Python schnittstelle enthalten, da Python eine moderne, weit verbreitete und vor allem gut lesbare Programmiersprache ist, die vielen Studenten und Studentinnen vertraut ist. Die gute lesbarkeit und klare Syntax sind für ein interaktives Framework wie dieses von Vorteil [EK17]. Das Framework soll für die Nutzung innerhalb der JupyterLab-Umgebung konzipiert sein, da dadurch eine gute Portabilität gewährleistet wird. Außerdem ist JupyterLab bereits in den Robotik-Übungen an der Fachhochschule Dortmund im Einsatz.

Es wird vorausgesetzt, dass Transformationen von Körpern im dreidimen-

sionalen Raum visualisiert werden können. Diese Visualisierungen sollen bevorzugt direkt innerhalb von JupyterLab erfolgen. Sollte das nicht möglich sein, wäre es möglich die Visuelle Darstellung in einem separaten Browserfenster vorzunehmen. Was nicht erwünscht ist, ist dass für die Visualisierung lokal ein Programm installiert werden muss, da das die Portabilität einschränken würde. Darüber hinaus soll das Framework so gestaltet sein, dass es die komfortable Erstellung und Anwendung von Übungsaufgaben unterstützt und vereinfacht. Zusätzlich wird gefordert, dass das Framework mit SymPy kompatibel ist, da das Framework in bereits viele schon existierende Notebooks mit Übungsaufgaben integriert werden könnte.

Das Framework soll grundlegende Konzepte der räumlichen Transformation abdecken. Dazu zählen insbesondere Rotationen, Eulerwinkel, Roll-Pitch-Yaw-Winkel sowie Quaternionen zur Darstellung und Analyse von Orientierungen im dreidimensionalen Raum. Darüber hinaus ist vorgesehen, auch einfache Modelle radgetriebener Roboter zu integrieren, um kinematische Zusammenhänge und Bewegungsabläufe anschaulich darzustellen.

Langfristig ist geplant, das Framework um Funktionalitäten zur Modellierung stationärer Roboter, insbesondere Knickarmroboter, zu erweitern. In diesem Zusammenhang sollen auch die Vorwärts- und Rückwärtstransformationen behandelt werden. Aufgrund des Umfangs und der Komplexität dieser Themen wird dieser Teil voraussichtlich im Rahmen einer späteren Bachelorarbeit realisiert und vertieft.

Zwar existieren bereits etablierte Werkzeuge wie beispielsweise Peter Corke's Robotics Toolbox for Python, welche eine Vielzahl an Funktionen für die Modellierung und Analyse robotischer Systeme bieten. Allerdings ist die Nutzung dieser Toolbox im Kontext von JupyterLab nur eingeschränkt möglich. Die dort verwendeten Visualisierungen basieren auf einem separaten Renderer

namens Swift, welcher ein eigenes Browserfenster zur Darstellung benötigt und somit nicht direkt in JupyterLab eingebettet ist [Cor23, S. 77]. Darüber hinaus ist die Toolbox sehr umfangreich und adressiert viele Konzepte, die für die Anforderungen und Inhalte dieses Projekts nicht von unmittelbarer Relevanz sind.

Eine maßgeschneiderte Lösung, die gezielt auf die Bedürfnisse und Themen des Lehrkonzepts an der Fachhochschule Dortmund zugeschnitten ist, bietet daher klare Vorteile. Sie erleichtert die Erstellung spezifischer Übungsaufgaben durch den Aufgabensteller, fördert die Anschaulichkeit durch eingebettete Visualisierungen für Lernende und trägt insgesamt zur Effizienz und Qualität der Lehre im Bereich der robotischen Transformationen bei.

6 Aufbau des Frameworks

6.1 System- und Softwareumgebung

Das Framework wurde für die Nutzung in der JupyterLab-Umgebung entwickelt, um eine plattformunabhängige und benutzerfreundliche Arbeitsumgebung zu bieten. Durch den Einsatz von Tools wie Poetry zur Verwaltung von Abhängigkeiten und Bibliotheken wie ipywidgets und pythreejs zur interaktiven 3D-Visualisierung wird eine stabile und effiziente Nutzung ermöglicht.

Abbildung 1 visualisiert das Zusammenspiel der einzelnen Softwarekomponenten innerhalb des Frameworks. Dabei wird deutlich, wie die verschiedenen Technologien – von der Serverbereitstellung über die Paketverwaltung bis hin zur 3D-Visualisierung und interaktiven Steuerung – ineinandergreifen, um eine stabile, plattformunabhängige und benutzerfreundliche Entwicklungsumgebung bereitzustellen.

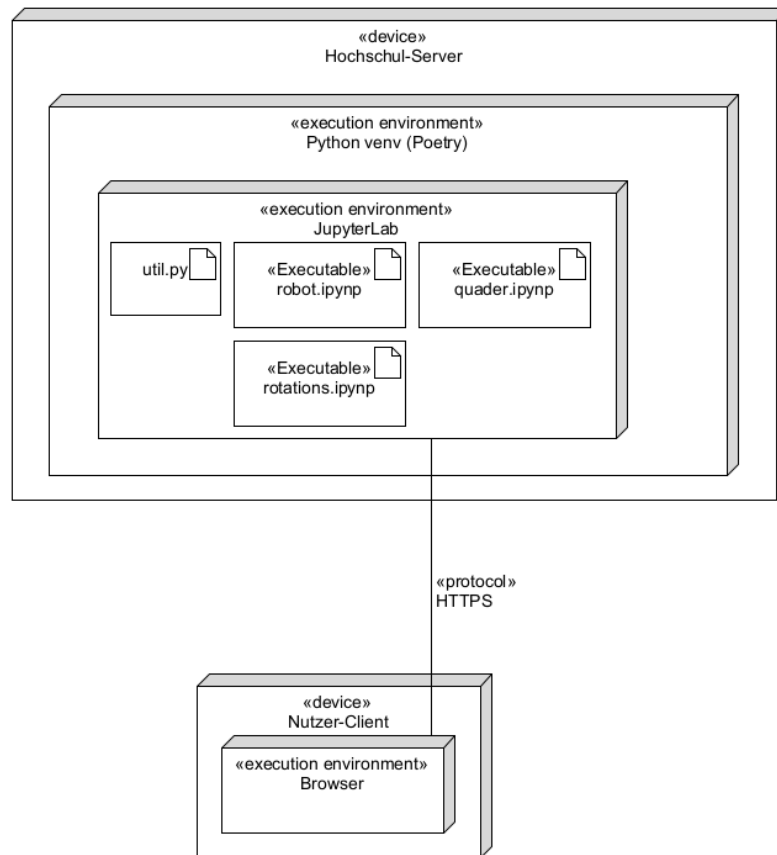


Abbildung 1: Verteilungsdiagramm des Frameworks

Das Framework ist, wie gefordert für die Nutzung innerhalb der JupyterLab-Umgebung ausgelegt. Die Fachhochschule Dortmund stellt hierfür einen eigenen JupyterLab-Server bereit, auf den sowohl über eine VPN-Verbindung als auch direkt im Hochschulnetz zugegriffen werden kann. Dadurch ist keine lokale Installation erforderlich, und die Arbeitsumgebung steht einheitlich auf verschiedenen Endgeräten zur Verfügung.

Um eine einfache Einrichtung des Frameworks auf dem Server zu gewährleisten,

ist es erforderlich, Versionskonflikte zwischen den verwendeten Abhängigkeiten zu vermeiden. Zu diesem Zweck wird das Werkzeug Poetry eingesetzt. Poetry ermöglicht die Verwaltung aller benötigten Python-Bibliotheken in festen Versionen und erstellt eine isolierte virtuelle Umgebung, in der alle Abhängigkeiten konsistent installiert sind. Dadurch wird sichergestellt, dass das Framework unabhängig von der Zielumgebung stabil und reproduzierbar ausgeführt werden kann.

Die Bibliothek ipywidgets wurde verwendet, um das Framework interaktiv zu gestalten. Mithilfe von Schiebereglern, Dropdown-Menüs und Checkboxes lassen sich Rotationen der gerenderten Objekte in weicher Echtzeit anpassen. Darüber hinaus ermöglichen die interaktiven Elemente eine benutzerfreundliche Steuerung weiterer Funktionen und tragen wesentlich zur intuitiven Bedienbarkeit des Frameworks bei.

Eine der zentralen Bibliotheken innerhalb des Projekts ist NumPy, eine weit verbreitete Python-Bibliothek für numerische Berechnungen. NumPy wird im Kontext dieses Projekts hauptsächlich für erweiterte mathematische Operationen im Bereich der linearen Algebra eingesetzt. Außerdem ist NumPy mit der Bibliothek SymPy, die für symbolische Mathematik verwendet wird gut kompatibel. Diese Kompatibilität ist ebenfalls eine Anforderung im Projekt. Obwohl NumPy eine leistungsfähige Grundlage für numerische Berechnungen bietet, enthält es keine Funktionen für spezielle geometrische Transformationen wie die Umwandlung von Euler-Winkeln in Quaternionen oder Rotationsmatrizen – oder deren Umkehrung. Daher wurde ergänzend die Bibliothek scipy.spatial verwendet, die solche Operationen effizient bereitstellt.

Darüber hinaus ist pythreejs als weitere wichtige Abhängigkeit integriert. Es handelt sich hierbei um eine Python-Wrapper-Bibliothek für die JavaScript-Bibliothek three.js, die auf WebGL basiert. pythreejs ermöglicht die Darstellung

und Interaktion mit 3D-Geometrien innerhalb eines WebGL-Renderers. Hierbei wird mithilfe von pythreejs JavaScript-Code generiert und im Hintergrund automatisch auf dem Client des Nutzers ausgeführt. Die Visualisierung erfolgt somit lokal im Webbrowser innerhalb des Notebooks, wo auch die Steuerung über Python stattfindet. Der Renderer kann daher nahtlos in der JupyterLab-Umgebung eingebettet und somit direkt in einem Jupyter Notebook visualisiert und gesteuert werden. Außerdem können auf diese Weise Geometrien mit Grafikbeschleunigung gerendert werden.

Durch den Einsatz dieser Technologien wird eine effiziente und plattformunabhängige Darstellung sowie Manipulation komplexer geometrischer Strukturen innerhalb des Frameworks ermöglicht.

Tabelle 1 gibt einen Überblick über die zentralen Softwaretechnologien, die für die Implementierung des Frameworks verwendet wurden. Diese Komponenten unterstützen verschiedene Aspekte wie numerische Berechnungen, interaktive Steuerung und 3D-Visualisierung.

Kategorie	Technologie	Beschreibung
Programmiersprache	Python	Hochsprachige, interpretierte Programmiersprache mit Fokus auf Lesbarkeit und Einfachheit – ideal für wissenschaftliches Rechnen, Datenanalyse und Prototyping.
Laufzeitumgebung	JupyterLab	Web-basierte Entwicklungsumgebung für interaktive Notebooks
Paketmanager	Poetry	Verwaltung von Abhängigkeiten und virtuellen Umgebungen
Mathe-Bibliothek	Numpy	Bibliothek für numerische Berechnungen und lineare Algebra
3D-Rendering	pythreejs	Python-Wrapper für three.js zur Darstellung von 3D-Geometrien in Notebooks
Interaktivität	ipywidgets	Erlaubt die Erstellung interaktiver GUI-Elemente wie Schieberegler, Dropdowns und Buttons in Jupyter-Notebooks

Tabelle 1: Überblick über die im Projekt verwendeten Softwaretechnologien

6.2 Framework-Library (Implementierungsdetails)

Die Bibliothek des Frameworks besteht aus einer zentralen Datei, `util.py`. In dieser Datei sind sämtliche Kernfunktionen implementiert, die zur Erstellung, Transformation und Animation geometrischer Formen benötigt werden. Darüber hinaus enthält `util.py` eine Reihe von Hilfsfunktionen, die unter anderem die Umrechnung zwischen verschiedenen Darstellungsformen von Rotationen ermöglichen sowie die Erstellung von Koordinatensystemen unterstützen.

Ergänzend dazu stellt die Bibliothek eine Klasse namens `Environment` bereit. `Environment` repräsentiert den Renderer mit einer Szene, die eine Kamera, Beleuchtung, ein Standardkoordinatensystem sowie ein Gitter enthält. Ein Objekt dieser Klasse kann vom Nutzer individuell angepasst werden. So lässt sich beispielsweise festlegen, ob – abweichend vom Standard – nicht die z-Achse, sondern die y-Achse nach oben zeigt. Auch das Gitter kann in seiner Auflösung verändert werden, indem die Größe der einzelnen Zellen angepasst wird – das Gitter kann somit feiner oder gröber dargestellt werden.

Zur Darstellung innerhalb eines Jupyter Notebooks kann ein `Environment`-Objekt über die dort übliche `display()`-Funktion gerendert werden. Darüber hinaus besteht die Möglichkeit, interaktive Widgets für beliebige Objekte hinzuzufügen, etwa zur Steuerung der Rotation über Schieberegler. Diese Widgets werden unterhalb des Renderers angezeigt und ermöglichen eine benutzerfreundliche Interaktion mit der Szene.

Zusätzlich enthält die Bibliothek eine Funktion `move(robot, vel, steps)`, mit der die Bewegung eines radgetriebenen Roboters in einer Ebene simuliert werden kann. Die Funktion nimmt als Parameter ein Roboterobjekt, eine Geschwindigkeitsangabe sowie die Anzahl der Zeitschritte entgegen und berechnet daraus die neue Position des Roboters im Raum.

6.3 Beispiel-Notebooks

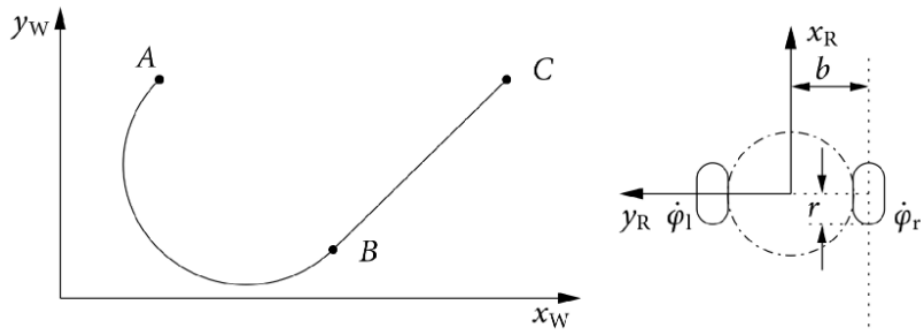
Das Framework enthält drei Beispiel-Notebooks, die die Anwendungsmöglichkeiten und Funktionalität praxisnah demonstrieren. Sie sollen dem Aufgabensteller als Orientierung dienen um mithilfe des Frameworks Aufgaben oder Umgebungen zum Lernen zu generieren.

6.3.1 robot.ipynb

Eines dieser Notebooks trägt den Namen robot.ipynb und dient der Veranschaulichung einer einfachen Steuerungsaufgabe. In diesem Notebook wird ein dreidimensional visualisierter Roboter mit Differentialantrieb dargestellt. Zusätzlich wird ein vorgegebener Pfad angezeigt, den der Roboter abfahren soll. Die Aufgabe besteht darin, die entsprechenden Antriebsgeschwindigkeiten zu berechnen und anschließend die Funktion `move(robot, vel, steps)` mit diesen Werten aufzurufen. Das Notebook Orientiert sich an einer Übungsaufgabe der Fachhochschule Dortmund im Modul Robotik aus dem Jahr 2024

Gegeben ist ein mobiler Roboter mit Differentialantrieb, der den im Weltkoordinatensystem abgebildeten Kurs von A nach C vorwärts abfährt. Der Kurs besteht aus einem Halbkreis zwischen den Punkten $A = (4, 9)$ und $B = (11, 2)$ und zwischen B und $C = (18, 9)$ aus einer Geraden. Die Positionen der Punkte sind in m angegeben.

[H]



Der Roboter soll den gesamten Kurs mit einer konstanten Lineargeschwindigkeit von $\dot{x}_R = 1 \frac{m}{s}$ abfahren. Beschleunigungs- und Verzögerungsvorgänge sollen vernachlässigt werden. Der Roboter hat einen halben Radabstand von $b = 0.5m$ und Radradius von $0,25m$.

[...]

Welche Geschwindigkeiten in Roboterkoordinaten ($v_R = \dot{x}_R = (\dot{x}_R, \dot{y}_R, \dot{\theta})^T$) ergeben sich für die beiden Abschnitte jeweils? [...]

Nach Ausführung der jeweiligen Zelle beginnt der Roboter mit der Bewegung. Die Nutzerin bzw. der Nutzer kann dabei direkt im Jupyter Notebook beobachten, ob der Roboter dem gewünschten Pfad korrekt folgt.

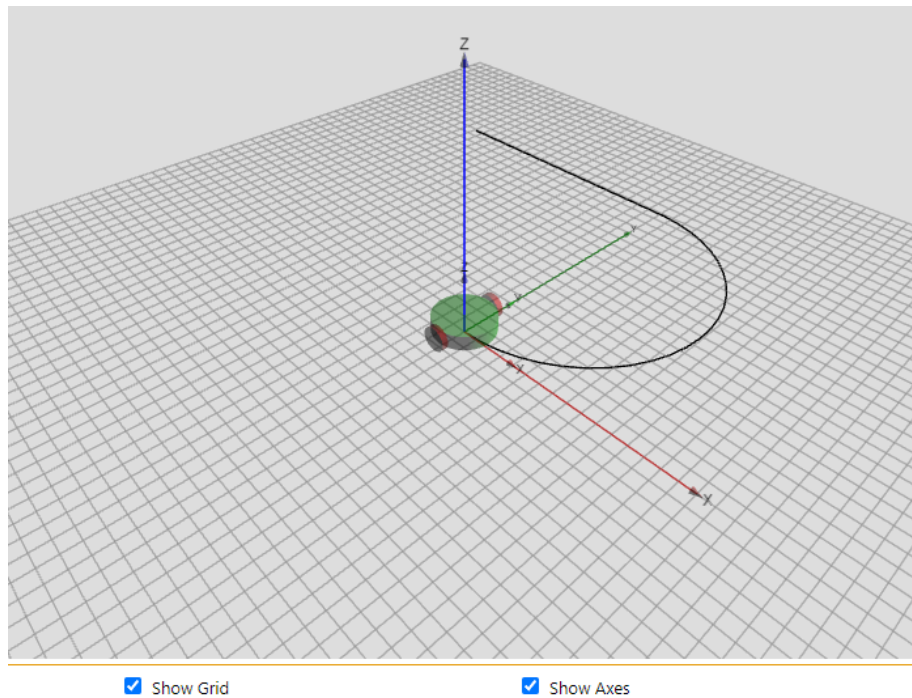


Abbildung 2: Verteilungsdiagramm des Frameworks (robot.ipynb während der Ausführung)

6.3.2 rotations.ipynb

Ein weiteres Beispiel-Notebook trägt den Namen rotations.ipynb. In diesem Notebook wird ein dreidimensionaler Würfel visualisiert, der vom Nutzer interaktiv transformiert werden kann.

Über integrierte Schieberegler lassen sich Translation, Rotation und Skalierung des Würfels in Echtzeit anpassen. Ziel dieses Notebooks ist es, die Möglichkeiten des Frameworks zur Darstellung und Simulation geometrischer Transformationen zu veranschaulichen.

Die direkte visuelle Rückmeldung innerhalb des Jupyter Notebooks ermöglicht es dem Lernenden, die Auswirkungen der jeweiligen Transformationen unmittelbar nachzuvollziehen und ein besseres Verständnis für deren Wirkung im dreidimensionalen Raum zu entwickeln.

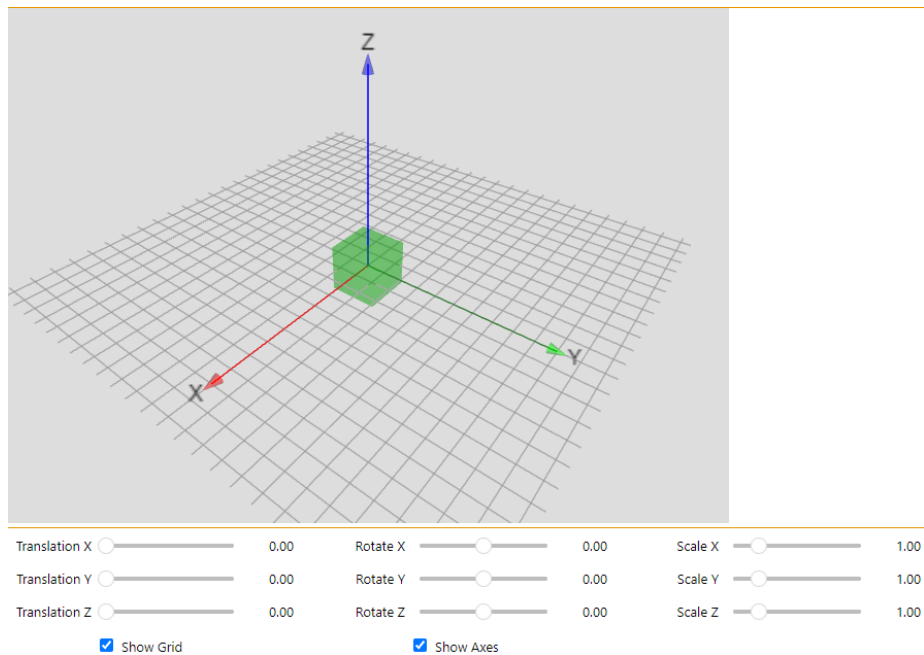


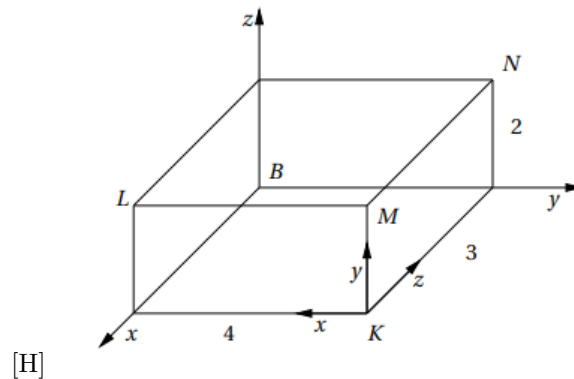
Abbildung 3: Würfel mit Schiebereglern für die Rotation (rotation.ipynb während der Ausführung)

6.3.3 quader.ipynb

das andere Beispiel-Notebook trägt den Namen quader.ipynb. Es Orientiert sich an einer Klausuraufgabe der Fachhochschule Dortmund im Modul Robotik aus dem Jahr 2010, in der ein Quader um die Achsen des Basis-Koordinatensystems rotiert, die jeweiligen Rotationsmatrizen aufgestellt und zu einer Gesamtrotati-

onsmatrix zusammengefasst werden sollen.

Gegeben ist ein quaderförmiger Körper mit den Kantenlängen $l_x = 3, l_y = 4$ und $l_z = 2$. Eine Quaderecke befindet sich, wie in der nachfolgenden Abbildung dargestellt, im Ursprung eines ortsfesten Koordinatensystems B . In der Quaderecke K ist ein körperfestes Koordinatensystem K angebracht.



Der Körper wird nun zunächst um den Winkel $\phi_z = 60^\circ$ um die z -Achse, dann um den Winkel $\phi_x = -180^\circ$ um die x -Achse und schließlich um den Winkel $\phi_y = -90^\circ$ um die y -Achse des ortsfesten Koordinatensystems B gedreht.

a) Stellen Sie die Rotationsmatrizen $R_x := R(x, -180^\circ)$, $R_y := R(y, -90^\circ)$ und $R_z := R(z, 60^\circ)$ auf.

b) Berechnen Sie die Rotationsmatrix R_G für die Gesamttrotation.

c) Welche Positionen nehmen nach der Drehung des Körpers die Körperecken M und N in Bezug auf das Koordinatensystem B ein?

d) Welche Orientierung, ausgedrückt als Rotationsmatrix, hat das Koordinatensystem K vor der Drehung der Körpers bezüglich der

Koordinatensystems B ?

e) Berechnen Sie die Eulerschen Winkel der Rotationsmatrix vor der Drehung.

f) Welche Orientierung, ausgedrückt als Rotationsmatrix, hat das Koordinatensystem K nach der Drehung des Körpers bezüglich des Koordinatensystems B ?

g) Berechnen Sie die Roll-Nick-Gier-Winkel der Rotationsmatrix von K nach der Drehung.

Das Notebook dient der Visuellen Simulation der Rotationen. Mit der Funktion `rotate_global_animated(mesh, angles, order)` wird der Quader zunächst wie in der Aufgabe beschrieben um die Achsen des Koordinatensystems B Rotiert. Diese Transformation wird Visuell dargestellt um ein Gefühl für die Rotation zu vermitteln und die Endposition aufzuzeigen. Anschließend werden die Matrizen als Variablen aufgestellt, zusammengefasst und an die Funktion `apply_rot_matrix_animated(mesh, rot_mat, speed, show_rot_mat)` übergeben, sodass die Rotation des Quaders anhand der Rotationsmatrix betrachtet und mit der vorrangegangenen Transformation verglichen werden kann. Dieses Notebook kann vom Aufgabensteller so modifiziert werden, dass die Matrizen von den Lernenden ausgefüllt werden müssen.

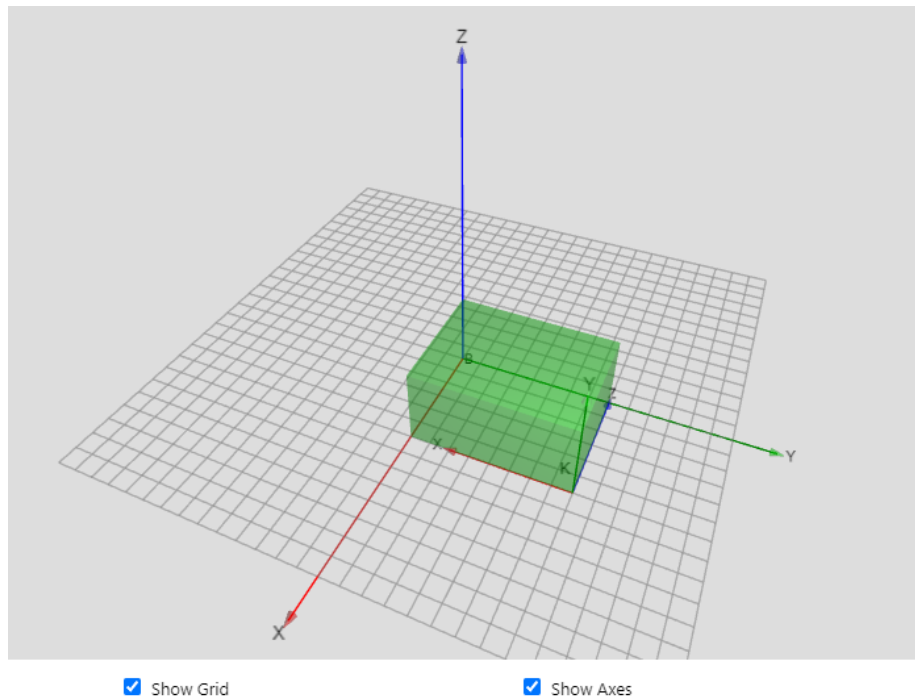


Abbildung 4: Quader aus der Klausuraufgabe (robot.ipynb während der Ausführung)

7 Entwicklungsprozess

Zu Beginn des Entwicklungsprozesses wurden verschiedene Technologien zur Darstellung und Manipulation von 3D-Grafiken miteinander verglichen. Dabei standen insbesondere OpenGL, WebGL, Babylon und ThreeJS im Fokus. Jede dieser Technologien bringt spezifische Vor- und Nachteile mit sich, die im Hinblick auf die Anforderungen des Projekts sorgfältig abgewogen wurden.

OpenGL überzeugte durch seine Plattformunabhängigkeit, da es auf ver-

schiedenen Betriebssystemen wie Windows, macOS und Linux eingesetzt werden kann. Dies ermöglichte Entwicklern, grafisch anspruchsvolle Anwendungen zu erstellen, die auf unterschiedlichen Systemen laufen, ohne dass spezifische Anpassungen nötig sind. Es war besonders beliebt in Bereichen wie der 3D-Grafik, CAD-Software und der Spieleentwicklung. Jedoch bringt OpenGL auch einige Einschränkungen mit sich. Eine davon ist, dass OpenGL lokal auf dem Gerät installiert werden muss, was zusätzliche Anforderungen an die Systemkonfiguration stellt und potenziell zu Kompatibilitätsproblemen führen kann. Zudem benötigt es ein eigenes Anwendungsfenster, was die Integration in Webanwendungen erheblich erschwert, da moderne Webanwendungen typischerweise ohne lokale Installation auskommen wollen [Gro25b].

WebGL stellt eine moderne, browserbasierte Alternative zu OpenGL dar, die auf denselben grundlegenden Prinzipien von OpenGL basiert. WebGL läuft direkt in aktuellen Webbrowsern, ohne dass zusätzliche Software installiert werden muss, was einen entscheidenden Vorteil hinsichtlich der Zugänglichkeit bietet. Durch die Nutzung von WebGL können Entwickler 3D-Inhalte in Webseiten einbinden, was besonders für interaktive Webanwendungen von Bedeutung ist. WebGL bietet ebenfalls eine plattformunabhängige Lösung, da es in allen gängigen Webbrowsern läuft, die HTML5 unterstützen. Der große Vorteil von WebGL liegt darin, dass Benutzer keine zusätzlichen Treiber oder Software installieren müssen. Allerdings ist WebGL relativ generisch und abstrakt aufgebaut. Dies bedeutet, dass viele der für eine vollständige 3D-Anwendung notwendigen Funktionen manuell implementiert werden müssen, was den Entwicklungsaufwand erheblich steigern kann. Zudem erfordert WebGL die Darstellung in einem separaten Browser-Tab, was die Integration von 3D-Grafiken in eine bestehende Webanwendung erschwert [Gro25a].

Babylon.js ist eine leistungsstarke 3D-Engine, die auf WebGL aufbaut und ei-

ne höher abstrahierte, fertige Lösung für die Entwicklung von 3D-Anwendungen bietet. Babylon ermöglicht die schnelle Erstellung komplexer 3D-Szenen, ohne dass tiefgehende Kenntnisse über das zugrunde liegende Rendering-System notwendig sind. Dies macht Babylon zu einer guten Wahl für Entwickler, die schnell funktionale 3D-Anwendungen erstellen möchten. Jedoch kann diese umfangreiche Funktionalität auch nachteilig sein, insbesondere wenn nur bestimmte Teilaspekte einer 3D-Anwendung benötigt werden. Babylon enthält beispielsweise ein vollständiges Physiksystem sowie ein Soundsystem, die für viele Projekte nicht erforderlich sind, was unnötige Komplexität hinzufügt. Ein weiterer Nachteil von Babylon.js ist, dass auch es ein eigenes Browser-Tab benötigt, um die Anwendung darzustellen, was die Benutzererfahrung und Integration in Webanwendungen einschränkt [Bab25a; Bab25c; Bab25b].

Three.js, ebenfalls auf WebGL basierend, bietet eine abstrahierte Schnittstelle zur Erstellung von 3D-Inhalten, geht jedoch einen Schritt weiter und bietet im Vergleich zu Babylon eine größere Flexibilität und Kontrolle. Three.js liegt etwas näher an der Low-Level-Programmierung und ermöglicht es Entwicklern, die Details des Renderings anzupassen und zu optimieren. Dies macht Three.js zu einem guten Kompromiss zwischen einfacher Nutzung und individueller Erweiterbarkeit, da es eine gute Balance zwischen abstrahierter Funktionalität und den Möglichkeiten zur Anpassung von 3D-Szenen bietet. Besonders für ein Projekt wie dieses, das im Rahmen einer Bachelorarbeit erweitert wird, erscheint diese Flexibilität von Vorteil, da noch nicht klar ist, welche spezifischen Features in die Anwendung aufgenommen werden sollen. Im Gegensatz zu Babylon ist Three.js leichtgewichtig und erlaubt eine genauere Anpassung an die Projektanforderungen. Im Vergleich zu OpenGL oder WebGL, die eine sehr niedriglevelige Herangehensweise bieten, überzeugt Three.js durch ein höheres Abstraktionsniveau, was die Entwicklung beschleunigt und vereinfacht. Wie bei den anderen

Lösungen erfolgt die Darstellung auch in einem eigenen Browser-Tab, was eine ähnliche Einschränkung wie bei WebGL und Babylon darstellt. Dennoch bietet Three.js aufgrund seiner Flexibilität und einfacheren Handhabung die beste Grundlage für dieses Projekt [Thr25].

Auf Basis dieser Überlegungen wurde im weiteren Verlauf des Entwicklungsprozesses ein einfacher Prototyp mithilfe der JavaScript-Bibliothek Three.js erstellt, um zu überprüfen, ob die grundlegenden Funktionalitäten, die zur Erfüllung der Projektanforderungen notwendig sind, in Three.js enthalten sind. Zu diesem Zweck wurde über den Node Package Manager (npm) das Paket live-server installiert, das es ermöglicht, eine HTML-Seite lokal zu hosten. Auf dieser Seite wurde der Prototyp eingebunden und interaktiv getestet.

Der Prototyp umfasst zunächst eine Kamera, ein Punktlicht sowie einen Würfel, die alle zu einer Szene hinzugefügt werden. Ergänzend dazu sind in der index.html-Datei Schieberegler integriert, mit denen die Rotation des Würfels über eine Benutzeroberfläche interaktiv gesteuert werden kann. Dies stellt die grundlegende Funktion des Prototyps dar: die interaktive Steuerung eines 3D-Objekts. Der Prototyp erfüllt damit die zentralen Anforderungen, indem er eine dynamische 3D-Visualisierung bietet, bei der der Benutzer in Echtzeit Einfluss auf die Darstellung nehmen kann. Diese erste Version des Prototyps ist ein wichtiger Schritt, um die Funktionalität und Flexibilität von Three.js für das Projekt zu validieren.

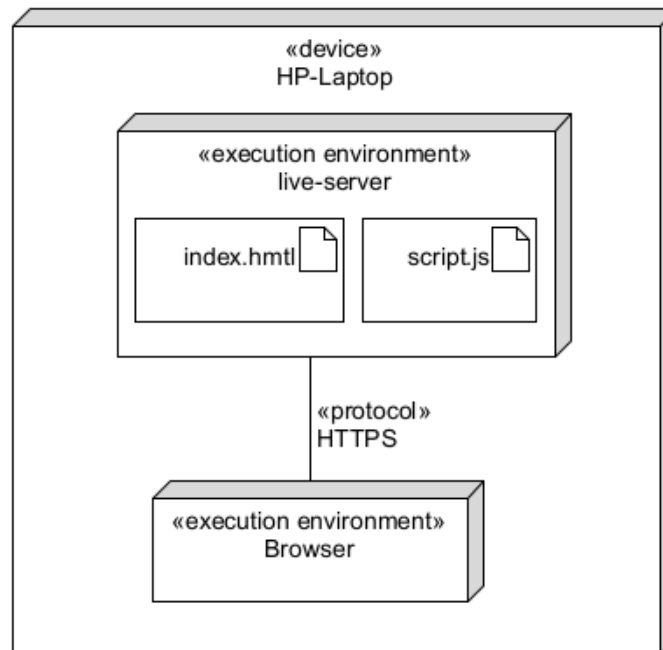


Abbildung 5: Verteilungsdiagramm des Frameworks

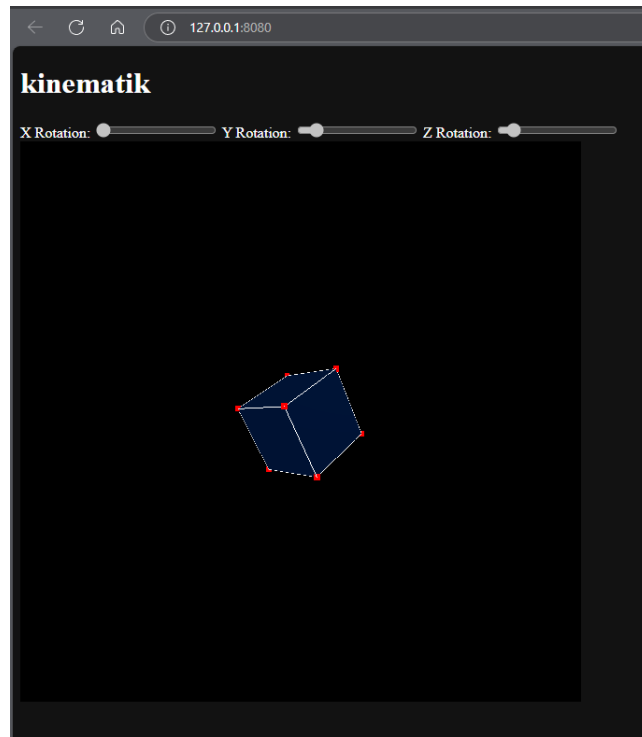


Abbildung 6: erster Prototyp mit threejs während der Ausführung im Browserfenster

Ein offensichtlicher Nachteil dieses Ansatzes ist jedoch, dass die visuelle Darstellung des Würfels in einem separaten Browser-Tab läuft, während die Python-Schnittstelle, die für das eigentliche Framework vorgesehen ist, in der JupyterLab-Umgebung und somit ebenfalls in einem eigenen Tab ausgeführt wird. Trotz dieser Einschränkung bestätigt der Prototyp, dass Three.js die benötigten Basisfunktionalitäten bereitstellt.

Der Nachteil, dass die visuelle Darstellung in einem separaten Browser-Tab erfolgt, lässt sich umgehen, indem man eine Python-basierte Schnittstelle verwendet, die direkt im JupyterLab-Umfeld arbeitet. Hierzu bietet sich die Bibliothek `pythreejs` an, ein Wrapper für Three.js, der eine Python-API zur Verfügung

stellt, welche im Hintergrund JavaScript-Code generiert und ausführt.

Pythreejs ist eine auf Jupyter Widgets basierende Notebook-Erweiterung, die es Jupyter ermöglicht, die WebGL-Fähigkeiten moderner Browser zu nutzen, indem sie Bindings zur JavaScript-Bibliothek Three.js bereitstellt. Durch die Einbettung in die jupyter-widgets-Infrastruktur lässt sich pythreejs nahtlos mit anderen interaktiven Werkzeugen innerhalb von Jupyter-Notebooks kombinieren, was die Entwicklung komplexer, interaktiver Visualisierungen erheblich erleichtert.

Die API von pythreejs orientiert sich dabei stark an der von Three.js, sodass vorhandene Dokumentationen und Ressourcen weitgehend übertragbar sind. Ein wesentlicher Unterschied liegt im sogenannten Render-Loop: Während Three.js typischerweise kontinuierlich Frames rendert, erlaubt pythreejs sowohl eine gezielte Einzelbilddarstellung per Aufruf als auch interaktive Darstellungen über spezielle Renderer-Klassen. Diese ermöglichen Funktionen wie OrbitControls (für das Schwenken, Zoomen und Drehen der Kamera) direkt innerhalb des Notebooks, ohne dass ein separater Browser-Tab erforderlich ist.

Nach weiteren Tests wurde deutlich, dass pythreejs im Vergleich zu three.js einige Einschränkungen mit sich bringt. Eine zentrale Erkenntnis war, dass sich das rotation-Attribut eines Mesh-Objekts in pythreejs nicht direkt verändern lässt. Stattdessen kann das quaternion-Attribut genutzt werden, um die Rotation zu steuern. Das macht es notwendig, Hilfsfunktionen zu implementieren, die Euler-Winkel in Quaternionen umrechnen. Dieser Schritt ist jedoch unproblematisch, da solche Funktionen ohnehin aus didaktischen Gründen Teil des Frameworks sein sollen.

Auf Basis dieser Erkenntnisse wurde ein erweiterter Prototyp mit pythreejs erstellt, bei dem die entwickelten Hilfsfunktionen aus der Datei util.py zum Einsatz kommen. Dieser Prototyp zeigte erfolgreich, dass die für das Framework

essenziellen Grundfunktionen – insbesondere die Animation und Rotation von Objekten – umsetzbar sind. Interaktive elemente konnten hier nicht mehr wie im vorherigen Prototyp durch HTML-Elemente hinzugefügt werden. Dafür wurde hier die Python-Bibliothek ipywidgets genutzt.

Da der Prototyp verschiedene numerische Berechnungen zur Transformation und Animation von Objekten durchführt, stellte sich die Frage, welche Bibliothek für die linearen Algebraoperationen am besten geeignet ist. Obwohl das Framework so gestaltet ist, dass es mit `sympy.Matrix`-Objekten umgehen kann, wurde in der konkreten Implementierung bewusst auf NumPy als Rechenbibliothek gesetzt. Der Grund dafür liegt in der deutlich besseren Effizienz von NumPy bei numerischen Berechnungen. Während SymPy primär auf symbolische Mathematik ausgelegt ist, wie das Umformen, Ableiten oder Vereinfachen mathematischer Ausdrücke, ist NumPy auf die schnelle numerische Verarbeitung konkreter Zahlenwerte optimiert.

Das zeigt sich besonders deutlich in der Rechengeschwindigkeit: In einem einfachen Test zur Matrixmultiplikation zweier 100x100-Matrizen benötigte NumPy nur etwa 0.0017 Sekunden, während SymPy für dieselbe Operation rund 9.7 Sekunden benötigte (siehe anhang). Der Unterschied liegt dabei nicht im mathematischen Verfahren selbst, sondern in der Art der Datenverarbeitung: NumPy arbeitet mit kompakten, typisierten Datenstrukturen und nutzt intern optimierte C-Implementierungen, während SymPy jeden Rechenschritt symbolisch verfolgt und daher deutlich langsamer ist [Num25; Sym25].

Da das Ziel des Frameworks ist, Transformationen numerisch auszuwerten und direkt auf reale Objekte anzuwenden, war eine symbolische Verarbeitung nicht erforderlich. Dennoch war eine der zentralen Anforderungen, dass das Framework mit `sympy.Matrix`-Objekten kompatibel ist. Daher wurde sichergestellt, dass solche Eingaben akzeptiert und automatisch in ein numerisch

geeignetes Format überführt werden, sodass auch bei Verwendung von SymPy eine korrekte und performante Ausführung gewährleistet bleibt – vorausgesetzt, die übergebenen `sympy.Matrix`-Objekte lassen sich auf konkrete Werte oder Annäherungen zurückführen.

Darauf aufbauend wurden die Notebooks `rotation.ipynb`, `quader.ipynb` und `robot.ipynb` entwickelt. Parallel dazu wurde die Bibliothek `util.py` schrittweise um zusätzliche Funktionen ergänzt, um komplexere Abläufe und Animationen realisieren zu können. Ein zentrales Prinzip bei der Animationslogik in `util.py` ist die Verwendung eines zeitbasierten Interpolationsschemas. Dabei wird die Rotation, Skalierung oder Translation zwischen zwei Zuständen schrittweise überführt. Der Ablauf folgt dabei einem einfachen Muster:

Algorithm 1 Interpolation einer Transformation

```

1: speed                                     ▷ Interpolationsgeschwindigkeit
2:  $t \leftarrow 0$ 
3:  $\Delta t \leftarrow 0.002$ 
4: while  $t \leq 1$  do
5:   mesh.transform  $\leftarrow \text{interpolate}(\text{old\_transform}, \text{new\_transform}, t)$ 
6:    $t \leftarrow t + \Delta t$ 
7:   wait( $1/\text{speed}$ )
8: end while
9: mesh.transform  $\leftarrow \text{interpolate}(\text{old\_transform}, \text{new\_transform}, 1)$ 

```

Dabei beschreibt t den Fortschritt der Interpolation von 0 bis 1, Δt ist die Schrittweite. In jedem Schritt wird eine neue Zwischen-rotation, position oder Skalierung berechnet und auf das Objekt angewendet, sodass eine flüssige Bewegung entsteht. Bei der praktischen Umsetzung des Algorithmus wurde die python library `time` für das warten verwendet. Es fiel auf, dass `time.sleep()` ab einem gewissen Schwellenwert (etwa 1ms) keine weiteren Verkürzungen der

Wartezeit bewirkt. Dies liegt vermutlich an dem overhead der Funktion. Auch wenn kleinere Werte übergeben werden, wird faktisch immer eine Mindestwartezeit eingehalten, die sich technisch nicht weiter reduzieren lässt. Dadurch war die Steuerung der Animationsgeschwindigkeit unterhalb dieser Grenze nicht mehr differenzierbar.

Zum Ende der Programmierarbeit hin wurde klar, welche Gemeinsamkeiten die Szenarien in `rotation.ipynb`, `quader.ipynb` und `robot.ipynb` und womöglich auch in zukünftigen Aufgabentypen haben. So wurde die Klasse `Environment` entworfen, die wiederkehrende Elemente in einer flexiblen und erweiterbaren Struktur zusammenfasst.

Die Klasse `Environment` kapselt die Erstellung und Verwaltung einer dreidimensionalen Szene. Dabei werden grundlegende Bestandteile wie Kamera, Lichtquellen, Gitter und Achsenobjekte automatisch erzeugt und zu einer `Scene`-Instanz zusammengefügt. Darüber hinaus integriert die Klasse ein interaktives `Renderer`-Element mit Kamera-Steuerung (`OrbitControls`), das direkt im Notebook angezeigt wird.

Um eine besonders einfache Erweiterbarkeit zu gewährleisten, wurden Methoden wie `add`, `add_widget` und `add_gizmo_controls` hinzugefügt, die beliebige Objekte und interaktive Steuerelemente zur Szene hinzufügen.

Die `Environment`-Klasse übernimmt somit eine zentrale Rolle innerhalb des Frameworks. Sie abstrahiert technische Details der Visualisierung und erlaubt es, sich auf die inhaltliche Modellierung von Bewegungsabläufen zu konzentrieren. Dadurch soll die Entwicklung und Darstellung von Aufgaben in den Notebooks `rotation.ipynb`, `quader.ipynb` und `robot.ipynb` vereinfacht werden.

Die Kombination aus `pythreejs`, den in `util.py` enthaltenen Transformationsfunktionen und der `Environment`-Klasse bildet somit ein Fundament für die weitere Entwicklung des Frameworks. Erweiterungen wie die Entwicklung auf-

gabenbezogener Rotationsszenarien oder die schrittweise Analyse komplexerer Bewegungsabläufe lassen sich auf dieser Basis modular und interaktiv umsetzen.

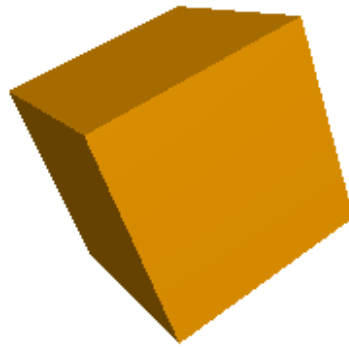


Abbildung 7: Notebook-Prototyp mit pythreejs während der Ausführung in JupyterLab

8 Evaluation

Im Rahmen dieses Projekts wurden viele funktionale und technische Anforderungen an das zu entwickelnde Visualisierungs-Framework definiert. Bei genauer Betrachtung lässt sich sagen, dass die meisten dieser Ziele erfolgreich erreicht wurden.

Ein besonders wichtiges Ziel war, das Framework so zu entwickeln, dass es

direkt in JupyterLab genutzt werden kann. Das hat gut funktioniert: Die Anwendung lässt sich problemlos in Jupyter Notebooks einbinden, was die Nutzung für Studierende unkompliziert und plattformunabhängig macht. Besonders praktisch ist dabei, dass die 3D-Visualisierung direkt im Notebook erscheint. Das macht die Bedienung und das Ausprobieren deutlich einfacher und anschaulicher.

Ein weiteres wichtiges Kriterium war die interaktive Bedienbarkeit. Durch die Verwendung von ipywidgets konnten benutzerfreundliche Bedienelemente wie Schieberegler und Dropdown-Menüs implementiert werden. Diese ermöglichen eine intuitive Steuerung von Parametern, wie zur Anpassung von Rotationen und erfüllen somit die Anforderung nach einer interaktiven Auseinandersetzung mit kinematischen Konzepten.

Die Kompatibilität mit SymPy, die symbolische Berechnungen und deren Integration in die Visualisierungen erlaubt, wurde ebenfalls sichergestellt. Die Funktionen in Util.py können problemlos Matrixobjekte der Sympy Bibliothek entgegennehmen.

Im inhaltlichen Fokus stand die Darstellung und Transformation von Rotationen im dreidimensionalen Raum. Die Umsetzung verschiedener Darstellungsformen wie Eulerwinkel, Roll-Pitch-Yaw-Winkel, Rotationsmatrizen und Quaternionen wurde erfolgreich realisiert. Die entsprechenden Umrechnungen zwischen diesen Darstellungen sowie deren visuelle Präsentation im Raum stellen eine große didaktische Stärke des Frameworks dar und decken die Anforderungen vollständig ab.

Auch die Simulation von radgetriebenen Robotern wurde erfolgreich integriert. Mit der Funktion `move(robot, vel, steps)` lassen sich einfache Bewegungsabläufe in der Ebene darstellen. Das erweitert das Framework sinnvoll über reine Rotationen hinaus.

Die Verwendung von Poetry zur Verwaltung von Abhängigkeiten sorgt darüber hinaus für eine stabile und konsistente Entwicklungsumgebung, was insbesondere für die langfristige Nutzung und Weiterentwicklung des Frameworks relevant ist.

Einziger Aspekt, der im aktuellen Projektstatus noch nicht vollständig umgesetzt wurde, ist die Modellierung stationärer Roboter (z.B. Knickarmroboter) samt Vorwärts- und Rückwärtskinematik. Allerdings wurde dies von Beginn an als zukünftige Erweiterung vorgesehen und fällt nicht in den unmittelbaren Umfang dieser Projektarbeit.

Es lässt sich feststellen, dass das Framework die definierten Anforderungen in nahezu allen Punkten erfüllt. Es bietet eine funktional umfangreiche, technisch stabile und didaktisch wertvolle Plattform zur Unterstützung der Lehre im Bereich der robotischen Kinematik. Die Grundlage für eine spätere Erweiterung um komplexere Robotermodelle wurde ebenfalls gelegt, womit auch die langfristigen Projektziele realistisch erscheinen.

9 Ausblick auf die Bachelorarbeit

Aufbauend auf den Ergebnissen dieses Projekts ergeben sich viele Möglichkeiten zur Weiterentwicklung, die sich gut im Rahmen einer Bachelorarbeit umsetzen lassen. Ein naheliegender nächster Schritt ist es, das bestehende Framework um Funktionen zur Modellierung stationärer Roboter zu erweitern. Ein Knickarmroboter würde sich dafür besonders gut eignen.

Der Fokus dieser Erweiterung soll auf der Umsetzung der Vorwärts- und Rückwärtskinematik liegen. Während die Vorwärtskinematik beschreibt, wie man aus bekannten Gelenkwinkeln die Position des Endeffektors berechnet, geht es bei der Rückwärtskinematik darum, herauszufinden, welche Gelenkstellungen nötig sind, um eine bestimmte Zielposition zu erreichen. Beide Verfahren sind

essenziell für die Steuerung und Planung von Roboterbewegungen und stellen im Vergleich zu den bisherigen Transformationen im Projekt eine sinnvolle und etwas komplexere Erweiterung dar.

Ein wichtiges Ziel der Bachelorarbeit ist es außerdem, das Framework auf dem JupyterLab-Server der Fachhochschule Dortmund bereitzustellen, sodass es für Studierende leicht zugänglich wird. Dadurch soll sichergestellt werden, dass die neuen Funktionen nicht nur theoretisch existieren, sondern auch direkt im Lehrbetrieb eingesetzt werden können. Die Idee ist, dass Studierende ohne zusätzliche Installationen direkt im Browser mit dem Framework arbeiten und damit verschiedene Robotermodelle und kinematische Aufgaben ausprobieren können.

Auch die bestehenden interaktiven Elemente sollen weiterentwickelt werden. Beispielsweise könnten bei der Rückwärtskinematik verschiedene Lösungsansätze visualisiert und direkt verglichen werden. So bleibt das Framework weiterhin intuitiv und visuell verständlich.

Die Bachelorarbeit zu diesem Thema ist eine gute Gelegenheit, das Framework inhaltlich und funktional weiterzuentwickeln und gleichzeitig einen praktischen Beitrag zur Lehre an der Fachhochschule Dortmund zu leisten.

Literatur

- [Bab25a] Project BabylonJS. *BabylonJS*. 2025. URL: <https://www.babylonjs.com/> (besucht am 30.04.2025).
- [Bab25b] Project BabylonJS. *BabylonJS Dokumentation Physics*. 2025. URL: <https://doc.babylonjs.com/features/featuresDeepDive/physics/> (besucht am 30.04.2025).
- [Bab25c] Project BabylonJS. *BabylonJS Specification*. 2025. URL: <https://www.babylonjs.com/specifications/> (besucht am 30.04.2025).
- [Cor23] Peter Corke. *Robotics, Vision and Control: Fundamental Algorithms in Python*. eng. Third edition. Bd. 146. Springer Tracts in Advanced Robotics. Cham: Springer International Publishing AG, 2023. ISBN: 3031064682.
- [EK17] Johannes Ernesti und Peter Kaiser. *Python 3 : das umfassende Handbuch*. ger. 5., aktualisierte Auflage. Rheinwerk Computing. Bonn: Rheinwerk Verlag, 2017. ISBN: 9783836258647.
- [Gro25a] Khronos Group. *Getting Started*. 2025. URL: https://www.khronos.org/webgl/wiki/Getting_Started (besucht am 02.05.2025).
- [Gro25b] Khronos Group. *The Industry's Foundation for High Performance Graphics*. 2025. URL: <https://www.khronos.org/opengl/> (besucht am 02.05.2025).
- [Gut+16] Ángel Gutiérrez u. a. *The Second Handbook of Research on the Psychology of Mathematics Education: The Journey Continues*. eng. 1. Aufl. Rotterdam: Springer, 2016. ISBN: 9463005609.
- [HLN12] Joachim Hertzberg, Kai Lingemann und Andreas Nüchter. *Mobiler Roboter : eine Einführung aus Sicht der Informatik*. ger. eXamen.press. Berlin [u.a: Springer, 2012. ISBN: 9783642017254.

- [Jup24] Project Jupyter. *JupyterLab Documentation*. 2024. URL: <https://jupyterlab.readthedocs.io/en/stable/> (besucht am 14. 04. 2025).
- [Kui99] J. B. Kuipers. *Quaternions and Rotation Sequences : A Primer with Applications to Orbits, Aerospace and Virtual Reality*. eng. Princeton, NJ: Princeton University Press, 1999. ISBN: 9780691211701.
- [Num25] Project Numpy. *what is numpy*. 2025. URL: <https://numpy.org/devdocs/user/whatisnumpy.html> (besucht am 03. 05. 2025).
- [SNS11] Roland Siegwart, Illah Reza Nourbakhsh und Davide Scaramuzza. *Introduction to autonomous mobile robots*. eng. 2. ed. Intelligent robotics and autonomous agents. Cambridge, Mass. [u.a: MIT Press, 2011. ISBN: 9780262015356.
- [Sym25] Project SymPy. *advanced usage*. 2025. URL: <https://docs.sympy.org/latest/explanation/best-practices.html#advanced-usage> (besucht am 03. 05. 2025).
- [Thr25] Project ThreeJS. *ThreeJS Manual Fundamentals*. 2025. URL: <https://threejs.org/manual/#en/fundamentals> (besucht am 30. 04. 2025).